Hybrid-NL2SVA

Part 1: Improving the Hybrid-NL2SVA pipeline

Part 2: An end-to-end Spec2SVA framework

Hybrid-NL2SVA Project

New York University

September 3, 2026

Hybrid-NL2SVA: Integrating RAG and Finetuning for LLM-based NL2SVA, MLCAD 2025.

NYU

Outline

Part I

The Hybrid-NL2SVA Pipeline

Why NL2SVA is hard

Hardware verification can take up to 70% of chip design time

SVAs are small checkers built into the design: they watch the hardware run and flag it the moment real behavior breaks a rule from the spec. Writing them by hand has two steps: (1) find the rules to check, in plain English, and (2) turn each rule into an SVA – step (2) is **NL2SVA**, and it is what this project is about.

Why a general AI model struggles.

- Not much SVA code in the data these models learned from
- SVA has timing words that look almost the same but mean different things:
  - `a |-> b`: b same cycle as a. `a |=> b`: b *1 cycle later*
  - `a ##2 b`: b *exactly* 2 cycles later. `a ##[1:3] b`: b *anywhere* in cycles 1–3
- Input descriptions can be written at very different abstraction levels of detail, for the same rule:
  - *high-level idea: "the handshake should work correctly"*
  - *some detail: "that ack correctly follows a request"*
  - *fully precise: "whenever req rises, ack must be asserted exactly 1 clock cycle later"*

  the first two need grounding first (Step 1, next slide)

Our approach: three steps

Never trust one AI answer

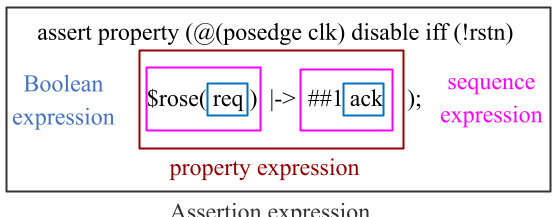
Step 1: rewrite the input into **OL-NL** (operator-level natural language) – name only real signals (e.g. `req`, `ack`, `clk`, `rstn`), and describe how they relate in plain words that match one specific operator, e.g. *"whenever req rises, ack must be asserted exactly 1 clock cycle later"* matches `$rose(req) |-> ##1 ack`, operator for operator, in plain English

Step 2: HybridRetrieval-Augmented Generation – give the model the right operator knowledge, then write a first answer

Step 3: use JasperGold to fix syntax errors; also parse the generated SVA into a syntax tree to automatically catch and fix mistakes in what it actually checks

The structure of an SVA

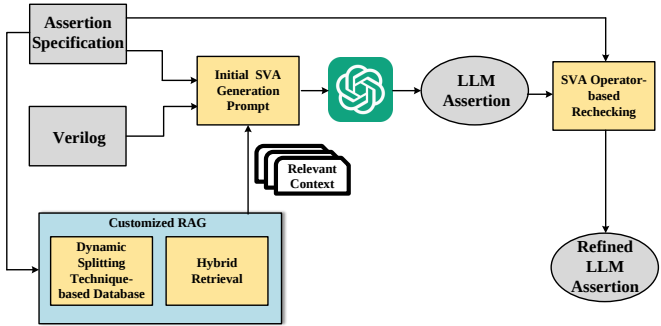
Four nested layers, each with its own operators – the source of most confusion



A Boolean expression (`req`) is wrapped by a sequence expression (`$rose(req) |-> ##1 ack`), wrapped by a property expression, wrapped by the assertion statement itself – getting any one layer's operator wrong (e.g. a sequence operator used where a property operator belongs) silently changes the property's meaning rather than raising a syntax error.

The original pipeline (MLCAD'25 paper)

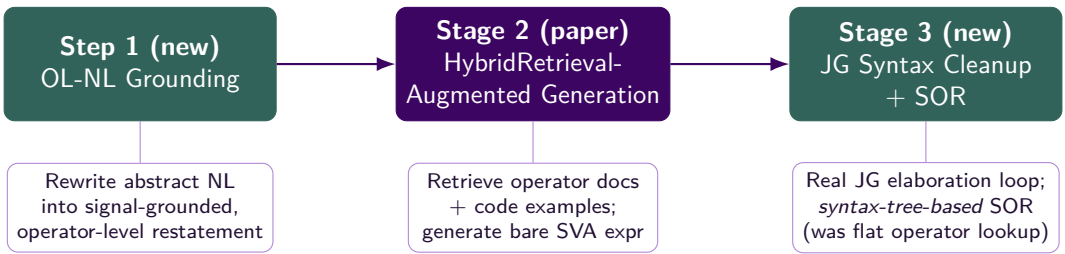
The customized RAG framework, from the MLCAD'25 paper



Customized RAG bundles two techniques: **dynamic splitting** (builds a code-centric retrieval database from SVA textbooks, preserving each code snippet with its surrounding explanation) and **HybridRetrieval** (next slide). Retrieved context enriches the Initial SVA Generation Prompt; the output is then passed through SVA Operator-based Rechecking (Stage 3) for a refined final assertion.

The pipeline today: three stages

Step 1 and Stage 3's SOR are both new – only Stage 2 is unchanged from the paper



Step 1 is only needed when the input isn't already signal-grounded. Stage 3's SOR now parses the candidate SVA into a syntax tree and rebuilds its meaning bottom-up, replacing the paper's flat per-operator lookup.

Stage 1 in detail: what is OL-NL?

Only needed when the input doesn't already name real signals

OL-NL = **operator-level natural language**: a plain-English sentence with two rules – (1) name *only* real signals from the design, never a made-up name, and (2) say exactly how those signals relate (e.g. "at the same time" or "N cycles later"); each such relation matches one specific SVA operator, but stays plain words, not code.

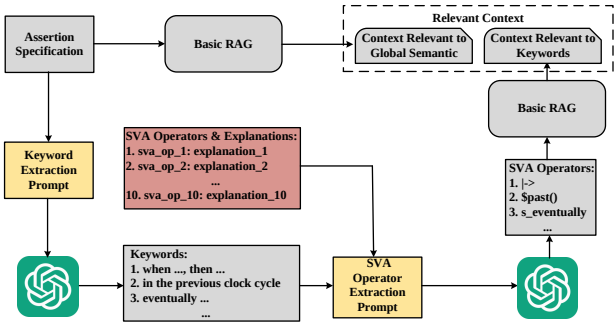
Worked example (same signals as the earlier SVA):

```
Plain description: that ack correctly follows a request.
  Real signals: req, ack, clk, rstn
OL-NL: Whenever req rises, then exactly 1 clock cycle
later, ack must be asserted.
```

Names only real signals, states the exact timing – matches `$rose(req) |-> ##1 ack` operator for operator, in plain English.

Stage 2 in detail: HybridRetrieval-augmented generation

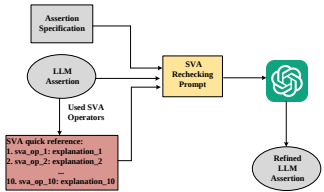
Two independent retrieval paths feed one generation call – from the MLCAD'25 paper



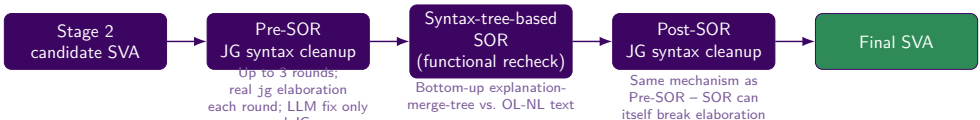
Path 1 (top): Basic RAG on the full assertion specification – global-semantic similarity against the code database. **Path 2 (bottom):** an LLM decomposes the description into keyword phrases (*Keyword Extraction Prompt*), maps each to the closest of 10 SVA operators (*SVA Operator Extraction Prompt*), then a second Basic RAG pass retrieves chunks specifically discussing those operators. Both paths' **Relevant Context** feeds the single generation call.

Stage 3 / SOR: then vs. now

As published (MLCAD'25 paper): flat, per-operator lookup



Today: full Pre-SOR / SOR / Post-SOR pipeline, syntax-tree-based

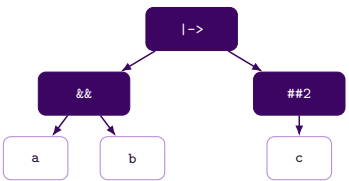


Inside SOR: the explanation-merge-tree

Parse to a syntax tree, merge bottom-up into NL, then compare against the original ask

LLM-generated SVA (Stage 2): `a && b |-> ##2 c`
Original ask – OL-NL (Stage 1): whenever a and b are both asserted, then c is asserted 3 cycles later

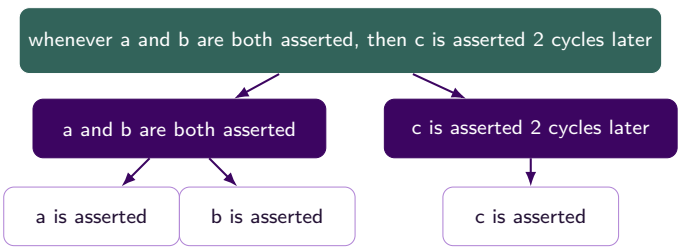
1. Parse into a syntax tree



root `|->` splits into `a && b`

and `##2 c`; each splits again

2. Merge that same tree bottom-up into NL



→ **3. Compare with the original ask above – mismatch found:** the tree's root says **2** cycles later, but the original ask says **3** – SOR flags just the ##2 node and revises it to ##3, not the whole property.

Benchmark & metrics

FVEval-Verified (wyt2000/FVEval-Verified): a human-expert-corrected version of FVEval's `nl2sva_human` and `nl2sva_machine` benchmarks.

- **nl2sva_human_verified** (73 rows) – expert-written ground-truth SVAs and NL descriptions

  e.g. "that if the current cycle is neither a jump operation nor immediately after a reset pulse, the counter must not overflow" → `(!jump_vld_d1 && !tb_reset_1_cycle_pulse_shadow) |-> (count <= max)`

- **nl2sva_machine_verified** (283 rows) – rule-generated SVAs, LLM-written NL descriptions; bare dummy testbenches, no reset signal

  e.g. "if both sig_G and sig_C are high and sig_A is high, then sig_I must not be high 5 clock cycles later" → `(sig_G && (sig_C && sig_A)) |-> ##5 !sig_I`

Comparison work: QiMeng-CodeV-SVA: Training Specialized LLMs for Hardware Assertion Generation via RTL-Grounded Bidirectional Data Synthesis, DAC 2026 – a model finetuned specifically for NL2SVA; shown alongside general models as a raw-output baseline.

Metrics (via a real JasperGold `prop_eq_checker`): **SC** – does the candidate elaborate against the real testbench on its own? **FM-strict** – formally proven full equivalence to the golden SVA.

Results (nl2sva_human_verified)

Our pipeline vs. raw model output vs. CodeV-SVA

Model	Configuration	SC	FM-strict	Δ
gpt-4o	Raw (0-shot)	94.5%	63.0%	
	+ Our pipeline	98.6%	**74.0%**	**+11.0**
DeepSeek-V4-Flash (thinking)	Raw (0-shot)	98.6%	76.7%	
	+ Our pipeline	100.0%	**86.3%**	**+9.6**
deepseek-chat (thinking off)	Raw (0-shot)	94.5%	54.8%	
	+ Our pipeline	100.0%	**74.0%**	**+19.2**
qwen3-8b (Aliyun)	Raw (base)	89.0%	35.6%	
	+ Our pipeline	91.8%	35.6%	+0.0
qwen3-14b (Aliyun)	Raw (base)	93.2%	49.3%	
	+ Our pipeline	91.8%	**65.8%**	**+16.5**
gpt-5	Raw (0-shot)	–	–	
	+ Our pipeline	**100.0%**	**82.2%**	–
CodeV-SVA-8B (DAC 2026, specialized)	–	97.3%	75.3%	–
CodeV-SVA-14B (DAC 2026, specialized)	–	93.2%	74.0%	–

Best per model family in **bold** (out of 73 rows). DeepSeek-V4-Flash + our pipeline (86.3%) already beats both CodeV-SVA sizes (75.3% / 74.0%), a model finetuned specifically for NL2SVA.

Results (nl2sva_machine_verified)

Our pipeline vs. raw model output vs. CodeV-SVA

Model	Configuration	SC	FM-strict	Δ
gpt-4o (+ our pipeline, no 0-shot run)	–	99.6%	84.8%	–
DeepSeek-V4-Flash (thinking)	Raw (0-shot)	100.0%	90.8%	
	+ Our pipeline	100.0%	**92.9%**	**+2.1**
deepseek-chat (thinking off)	Raw (0-shot)	97.5%	78.4%	
	+ Our pipeline	98.9%	**85.5%**	**+7.1**
qwen3-8b (Aliyun)	Raw (base)	87.6%	51.2%	
	+ Our pipeline	88.7%	**60.8%**	**+9.6**
qwen3-14b (Aliyun)	Raw (base)	91.9%	65.7%	
	+ Our pipeline	95.1%	**81.6%**	**+15.9**
CodeV-SVA-8B (DAC 2026, specialized)	–	96.5%	83.0%	–
CodeV-SVA-14B (DAC 2026, specialized)	–	97.2%	80.9%	–

Best per model family in **bold** (out of 283 rows). DeepSeek-V4-Flash + our pipeline (92.9%) is the strongest result overall, ahead of both CodeV-SVA sizes (83.0% / 80.9%). gpt-5's machine run was interrupted (45/283) and is omitted.

Part II

End-to-End Spec2SVA

From ready-made benchmarks to real designs

What Part I evaluated:
NL2SVA on FVEval-Verified – properties and RTL are already picked out and ready to use, one property per row, always paired with a golden (known-correct) SVA.

What real verification looks like:
a *specification document* (PDF, dozens of pages) and a *multi-file RTL design* – no pre-extracted property list, no golden SVA to check against.

Goal: close the loop – *Spec2NL* (turn a spec + RTL into a list of natural-language verification properties) feeding directly into *our* NL2SVA pipeline, with JasperGold verifying the result end to end.

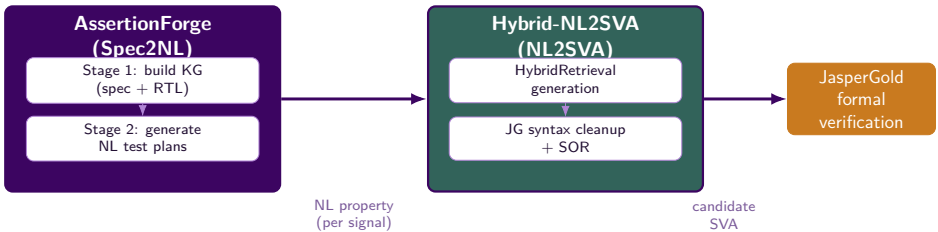
AssertionForge (NVlabs, LAD 2025)

github.com/NVlabs/AssertionForge — arXiv:2503.19174

- **Stage 1 – Knowledge Graph Construction:** builds a joint KG from the spec (via GraphRAG, domain-customized entity-extraction prompt) and the RTL (`pyverilog`-based structural analysis), then fuses the two
- **Stage 2 – Test Plan Generation:** for each architectural signal, retrieves KG-guided context (RAG chunks + graph-path analysis), prunes with an LLM judge, and generates natural-language verification *plans*

Stage 1 + Stage 2 together = **Spec2NL**: spec PDF + RTL directory → a list of natural-language properties, each grounded in a specific signal and its role in the design.

Spec2NL + NL2SVA = end-to-end Spec2SVA



NL2SVA runs with **the same settings as the nl2sva_machine_verified evaluation**, minus Step 1 (AssertionForge's plans are already signal-grounded) – plus support for a real design's actual clock name (rarely just `clk`).

Case study: UART

AssertLLM's spec + RTL

Design.

- 6 Verilog modules: `uart2bus_top` (real top), `uart_top`, `baud_gen`, `uart_rx`, `uart_tx`, `uart_parser`
- 246 total signals; 18 architectural (interface) signals across the hierarchy
- Real clock port name: `clock` (not `clk`)

Spec2NL output.

- Knowledge graph: 1,371 nodes, 1,620 edges (spec + RTL fused)
- **323** natural-language verification properties generated (all 18 signals)
- QiMeng-CodeV-SVA's own Table 5 reports **265** SVAs for UART (GPT-4o Spec2NL + GPT-4o NL2SVA) – same order of magnitude

A real gap found & fixed along the way: upstream's own signal list example only listed 2 of the 18 real signals, and its LLM-client code shipped as an empty "add your own client" stub – both needed fixing before Stage 1/2 ran end to end.

Sample generated NL properties

- **baud_clk:** "that after a reset condition, baud_clk will resume toggling without skipping cycles based on the previously set value of baud_freq"
- **baud_freq:** "that baud_freq maintains a constant value when the `global_clock_freq` and `baud_rate` inputs remain unchanged over 20 clock cycles"
- **int_req / int_gnt:** bus request/grant handshake properties derived from path analysis between `uart2bus_top`'s internal bus interface signals

Example NL2SVA output (`clock_signal=clock`, no `disable iff`, matching the machine-dataset config):

```
asrt: assert property (@(posedge clock)
    (new_tx_data && !tx_busy) |-> tx_data
);
```

Validated: same 44 properties, ungrounded vs. grounded

`baud_clk/baud_freq` **pilot – the exact same starting ideas, matched one to one**

Same 44 properties, different generation	**n**	**#SynC**	**#Proven**
Raw ideas + gpt-4o 0-shot baseline (no pipeline)	44	30 (68.2%)	15 (34.1%)
Raw ideas + CodeV-SVA-8B (no pipeline)	44	18 (40.9%)	12 (27.3%)
Raw ideas + CodeV-SVA-14B (no pipeline)	44	21 (47.7%)	13 (29.5%)
Grounded (Step 2 + filter) + full Hybrid-NL2SVA	**44**	**40 (90.9%)**	**24 (54.5%)**

Same 44 properties both ways (matched one to one) – grounding + the full pipeline is **+22.7pp #SynC, +20.4pp #Proven** over the best ungrounded baseline; a specialized NL2SVA model alone (CodeV-SVA) can't make up for bad, made-up signal names the way grounding does. *Scope: `baud_clk/baud_freq` only (2/18 signals) – a small test before the remaining 16 signals and a full re-run.*

Next steps

- Run the two-step generation + mechanical filter on the remaining 16 signals and fully regenerate `nl_plans_uart.txt`
- Extend the pilot to the remaining 4 designs from QiMeng-CodeV-SVA's Table 5 (APB, ETHMAC, OPENMSP430, SOCKIT)
- Swap AssertionForge's own NL2SVA backend for a head-to-head comparison at the *same* Spec2NL properties
- Try DeepSeek-V4-Flash / qwen3 as AssertionForge's own Spec2NL backend, not just gpt-4o
- Look into why #SynC (69.7%) trails #SVA – categorize the elaboration failures the same way Part I's row-by-row diffing did

Thank you